

RAR 5.0 archive format

Here we describe basic data structures of archive format introduced in RAR 5.0. If you need information about algorithms or more detailed information on data structures, please use UnRAR source code.

Contents

- [Data types](#)
 - [vint](#)
 - [byte, uint16, uint32, uint64](#)
 - [Variable length data](#)
 - [Hexadecimal values](#)
- [General archive structure](#)
 - [General archive block format](#)
 - [General extra area format](#)
 - [General archive layout](#)
- [Archive blocks](#)
 - [Self-extracting module \(SFX\)](#)
 - [RAR 5.0 signature](#)
 - [Archive encryption header](#)
 - [Main archive header](#)
 - [Extra record types](#)
 - [Locator record](#)
 - [Metadata record](#)
 - [File header and service header](#)
 - [Extra record types](#)
 - [File encryption record](#)
 - [File hash record](#)
 - [File time record](#)
 - [File version record](#)
 - [File system redirection record](#)
 - [Unix owner record](#)
 - [Service data record](#)
 - [End of archive header](#)
- [Service headers](#)
 - [Archive comment header](#)
 - [Quick open header](#)

Data types

vint

Variable length integer. Can include one or more bytes, where lower 7 bits of every byte contain integer data and highest bit in every byte is the continuation flag. If highest bit is 0, this is the last byte in sequence. So first byte contains 7 least significant bits of integer and continuation flag. Second byte, if present, contains next 7 bits and so on.

Currently RAR format uses vint to store up to 64 bit integers, resulting in 10 bytes maximum. This value may be increased in the future if necessary for some reason.

Sometimes RAR needs to pre-allocate space for vint before knowing its exact value. In such situation it can allocate more space than really necessary and then fill several leading bytes with 0x80 hexadecimal, which means 0 with continuation flag set.

byte, uint16, uint32, uint64

Byte, 16-, 32-, 64- bit unsigned integer in little endian format.

Variable length data

We use ellipsis ... to denote variable length data areas.

Hexadecimal values

We use 0x prefix to define hexadecimal values, such as 0xf000

General archive structure

General archive block format

Field	Size	Description
Header CRC32	uint32	CRC32 of header data starting from <i>Header size</i> field and up to and including the optional extra area.
Header size	vint	Size of header data starting from <i>Header type</i> field and up to and including the optional extra area. This field must not be longer than 3 bytes in current implementation, resulting in 2 MB maximum header size.
Header type	vint	Type of archive header. Possible values are: 1 Main archive header. 2 File header. 3 Service header. 4 Archive encryption header. 5 End of archive header.
Header flags	vint	Flags common for all headers: 0x0001 Extra area is present in the end of header. 0x0002 Data area is present in the end of header. 0x0004 Blocks with unknown type and this flag must be skipped when updating an archive. 0x0008 Data area is continuing from previous volume. 0x0010 Data area is continuing in next volume. 0x0020 Block depends on preceding file block. 0x0040 Preserve a child block if host block is modified.
Extra area size	vint	Size of extra area. Optional field, present only if 0x0001 header flag is set.
Data size	vint	Size of data area. Optional field, present only if 0x0002 header flag is set.
...	...	Fields specific for current block type. See concrete block types description for details.
Extra area	...	Optional area containing additional header fields, present only if 0x0001 header flag is set.
Data area	vint	Optional data area, present only if 0x0002 header flag is set. Used to store large data amounts, such as compressed file data. Not counted in <i>Header CRC</i> and <i>Header size</i> fields.

General extra area format

Extra area can include one or more records having the following format:

Size	vint	Size of record data starting from <i>Type</i> .
------	------	---

Type	vint	Record type. Different archive blocks have different associated extra area record types. Read the concrete archive block description for details. New record types can be added in the future, so unknown record types need to be skipped without interrupting an operation.
Data	...	Record dependent data. May be missing if record consists only from size and type.

General archive layout

```

Self-extracting module (optional)
RAR 5.0 signature
Archive encryption header (optional)
Main archive header
Archive comment service header (optional)

File header 1
Service headers (NTFS ACL, streams, etc.) for preceding file (optional).
...
File header N
Service headers (NTFS ACL, streams, etc.) for preceding file (optional).

Recovery record (optional).
End of archive header.

```

Archive blocks

Self-extracting module (SFX)

Any data preceding the archive signature. Self-extracting module size and contents is not defined. At the moment of writing this documentation RAR assumes the maximum SFX module size to not exceed 1 MB, but this value can be increased in the future.

RAR 5.0 signature

RAR 5.0 signature consists of 8 bytes: 0x52 0x61 0x72 0x21 0x1A 0x07 0x01 0x00. You need to search for this signature in supposed archive from beginning and up to maximum SFX module size. Just for comparison this is RAR 4.x 7 byte length signature: 0x52 0x61 0x72 0x21 0x1A 0x07 0x00.

Archive encryption header

Header CRC32	uint32	
Header size	vint	
Header type	vint	4
Header flags	vint	Flags common for all headers
Encryption version	vint	Version of encryption algorithm. Now only 0 version (AES-256) is supported.
Encryption flags	vint	0x0001 Password check data is present.
KDF count	1 byte	Binary logarithm of iteration number for PBKDF2 function. RAR can refuse to process KDF count exceeding some threshold. Concrete value of threshold is a version dependent.
Salt	16 bytes	Salt value used globally for all encrypted archive headers.

Check value	12 bytes	Value used to verify the password validity. Present only if 0x0001 encryption flag is set. First 8 bytes are calculated using additional PBKDF2 rounds, 4 last bytes is the additional checksum. Together with the standard header CRC32 we have 64 bit checksum to reliably verify this field integrity and distinguish invalid password and damaged data. Further details can be found in UnRAR source code.
-------------	----------	--

This header is present only in archives with encrypted headers. Every next header after this one is started from 16 byte AES-256 initialization vector followed by encrypted header data. Size of encrypted header data block is aligned to 16 byte boundary.

Main archive header

Header CRC32	uint32	
Header size	vint	
Header type	vint	1
Header flags	vint	Flags common for all headers
Extra area size	vint	Size of extra area. Optional field, present only if 0x0001 header flag is set.
Archive flags	vint	0x0001 Volume. Archive is a part of multivolume set. 0x0002 Volume number field is present. This flag is present in all volumes except first. 0x0004 Solid archive. 0x0008 Recovery record is present. 0x0010 Locked archive.
Volume number	vint	Optional field, present only if 0x0002 archive flag is set. Not present for first volume, 1 for second volume, 2 for third and so on.
Extra area	...	Optional area containing additional header fields, present only if 0x0001 header flag is set.

Extra area of main archive header can contain following record types

Type	Name	Description
0x01	Locator	Contains positions of different service blocks, so they can be accessed quickly, without scanning the entire archive. This record is optional. If it is missing, it is still necessary to scan the entire archive to verify presence of service blocks.
0x02	Metadata	Optional record storing archive metadata, which includes archive original name and time.

Locator record

Size	vint	
Type	vint	1
Flags	vint	0x0001 Quick open record offset is present. 0x0002 Recovery record offset is present.
Quick open offset	vint	Distance from beginning of quick open service block to beginning of main archive header. Present only if 0x0001 flag is set. If equal to 0, should be ignored. It can be set to zero if preallocated space was not enough to store the resulting offset.
Recovery record offset	vint	Distance from beginning of recovery record service block to beginning of main archive header. Present only if 0x0002 flag is set. If equal to 0, should be ignored. It can be set to zero if preallocated space was not enough to store the resulting offset.

Metadata record

Size	vint	
Type	vint	2
Flags	vint	0x0001 Archive name is present. 0x0002 Archive original creation time is present. 0x0004 Use Unix time if 1, Windows FILETIME if 0. 0x0008 Defines Unix time as nanoseconds since 1970-01-01 if 1 and as seconds since 1970-01-01 if 0.
Name length	vint	Original archive name length. Present if flag 0x0001 is set.
Name	? bytes	Variable length field containing <i>Name length</i> bytes of original archive name in UTF-8 format. Present if flag 0x0001 is set. In general the trailing zero is missing, but if name length is changed while archiving, one or more trailing zeroes can be present as result of size overprovisioning. In this case the name shall be truncated at the first trailing zero. Typically it happens in the first volume in multivolume set. In rare case if initially reserved buffer size is not enough to store the resulting name, the first byte of this buffer is set to zero. It indicates that no name is stored even if its length is set to non-zero.
Time	4 or 8 bytes	Original archive creation time either in 8 bytes Windows FILETIME format if flag 0x0004 is 0, or in Unix format if 0x0004 is 1. Unix format can be 4 bytes seconds since 1970-01-01 if flag 0x0008 is 0 or 8 bytes nanoseconds since 1970-01-01 if 0x0008 is 1.

File header and service header

These two header types use the similar data structure, so we describe them both here.

Header CRC32	uint32	
Header size	vint	
Header type	vint	2 for file header, 3 for service header
Header flags	vint	Flags common for all headers
Extra area size	vint	Size of extra area. Optional field, present only if 0x0001 header flag is set.
Data size	vint	Size of data area. Optional field, present only if 0x0002 header flag is set. For file header this field contains the packed file size.
File flags	vint	Flags specific for these header types: 0x0001 Directory file system object (file header only). 0x0002 Time field in Unix format is present. 0x0004 CRC32 field is present. 0x0008 Unpacked size is unknown. If flag 0x0008 is set, unpacked size field is still present, but must be ignored and extraction must be performed until reaching the end of compression stream. This flag can be set if actual file size is larger than reported by OS or if file size is unknown such as for all volumes except last when archiving from stdin to multivolume archive.
Unpacked size	vint	Unpacked file or service data size.
Attributes	vint	Operating system specific file attributes in case of file header. Might be either used for data specific needs or just reserved and set to 0 for service header.
mtime	uint32	File modification time in Unix time format. Optional, present if 0x0002 file flag is set.

Data CRC32	uint32	CRC32 of unpacked file or service data. For files split between volumes it contains CRC32 of file packed data contained in current volume for all file parts except the last. Optional, present if 0x0004 file flag is set.
Compression information	vint	Lower 6 bits (0x003f mask) contain the version of compression algorithm, resulting in possible 0 - 63 values. Currently values 0 and 1 are possible. Version 0 archives can be unpacked by RAR 5.0 and newer. Version 1 archives can be unpacked by RAR 7.0 and newer. 7th bit (0x0040) defines the solid flag. If it is set, RAR continues to use the compression dictionary left after processing preceding files. It can be set only for file headers and is never set for service headers. Bits 8 - 10 (0x0380 mask) define the compression method. Currently only values 0 - 5 are used. 0 means no compression. Bits 11 - 15 (0x7c00) specify the minimum dictionary size required to extract data. If we define these bits as N, the dictionary size is 128 KB * 2^N. So value 0 means 128 KB, 1 - 256 KB, ..., 15 - 4096 MB, ..., 19 - 64 GB. 23 means 1 TB, which is the theoretical maximum allowed by this field. Actual compression and decompression implementation might have a lower limit. Values above 15 are used only if compression algorithm version is 1. Bits 16 - 20 (0xf8000) are present only if version of compression algorithm is 1. Value in these bits is multiplied to the dictionary size in bits 11 - 15 and divided by 32, the result is added to dictionary size. It allows to specify up to 31 intermediate dictionary sizes between neighbouring power of 2 values. Bit 21 (0x100000) is present only if version of compression algorithm is 1. It indicates that even though the dictionary size flags are in version 1 format, the actual compression algorithm is version 0. It is helpful when we append version 1 files to existing version 0 solid stream and need to increase the dictionary size for version 0 files not touching their compressed data.
Host OS	vint	Type of operating system used to create the archive. 0x0000 Windows. 0x0001 Unix.
Name length	vint	File or service header name length.
Name	? bytes	Variable length field containing <i>Name length</i> bytes in UTF-8 format without trailing zero. For file header this is a name of archived file. Forward slash character is used as the path separator both for Unix and Windows names. Backslashes are treated as a part of name for Unix names and as invalid character for Windows file names. Type of name is defined by <i>Host OS</i> field. If Unix file name contains any high ASCII characters which cannot be correctly converted to Unicode and UTF-8, we map such characters to 0xE080 - 0xE0FF private use Unicode area and insert 0xFFFF Unicode non-character to resulting string to indicate that it contains mapped characters, which need to be converted back when extracting. Concrete position of 0xFFFF is not defined, we need to search the entire string for it. Such mapped names are not portable and can be correctly unpacked only on the same system where they were created. For service header this field contains a name of service header. Now the following names are used: CMT Archive comment QQ Archive quick open data ACL NTFS file permissions

		STM NTFS alternate data stream RR Recovery record
Extra area	...	Optional area containing additional header fields, present only if 0x0001 header flag is set.
Data area	vint	Optional data area, present only if 0x0002 header flag is set. Store file data in case of file header or service data for service header. Depending on the compression method value in <i>Compression information</i> can be either uncompressed (compression method 0) or compressed.

File and service headers use the same types of extra area records:

Type	Name	Description
0x01	File encryption	File encryption information.
0x02	File hash	File data hash.
0x03	File time	High precision file time.
0x04	File version	File version number.
0x05	Redirection	File system redirection.
0x06	Unix owner	Unix owner and group information.
0x07	Service data	Service header data array.

File encryption record

This record is present if file data is encrypted.

Size	vint	
Type	vint	0x01
Version	vint	Version of encryption algorithm. Now only 0 version (AES-256) is supported.
Flags	vint	0x0001 Password check data is present. 0x0002 Use tweaked checksums instead of plain checksums. If flag 0x0002 is present, RAR transforms the checksum preserving file or service data integrity, so it becomes dependent on encryption key. It makes guessing file contents based on checksum impossible. It affects both data CRC32 in file header and checksums in file hash record in extra area.
KDF count	1 byte	Binary logarithm of iteration number for PBKDF2 function. RAR can refuse to process KDF count exceeding some threshold. Concrete value of threshold is version dependent.
Salt	16 bytes	Salt value to set the decryption key for encrypted file.
IV	16 bytes	AES-256 initialization vector.
Check value	12 bytes	Value used to verify the password validity. Present only if 0x0001 encryption flag is set. First 8 bytes are calculated using additional PBKDF2 rounds, 4 last bytes is the additional checksum. Together with the standard header CRC32 we have 64 bit checksum to reliably verify this field integrity and distinguish invalid password and damaged data. Further details can be found in UnRAR source code.

File hash record

Only the standard CRC32 checksum can be stored directly in file header. If other hash is used, it is stored in this extra area record:

Size	vint	
Type	vint	0x02
Hash type	vint	0x00 BLAKE2sp hash function.
Hash data	? bytes	32 bytes of BLAKE2sp for 0x00 hash type.

For files split between volumes it contains a hash of file packed data contained in current volume for all file parts except the last. For files not split between volumes and for last parts of split files it contains an unpacked data hash.

File time record

This record is used if it is necessary to store creation and last access time or if 1 second precision of Unix mtime stored in file header is not enough:

Size	vint	
Type	vint	0x03
Flags	vint	0x0001 Time is stored in Unix time_t format if this flags is set and in Windows FILETIME format otherwise 0x0002 Modification time is present 0x0004 Creation time is present 0x0008 Last access time is present 0x0010 Unix time format with nanosecond precision
mtime	uint32 or uint64	Modification time. Present if 0x0002 flag is set. Depending on 0x0001 flag can be in Unix time_t or Windows FILETIME format.
ctime	uint32 or uint64	Creation time. Present if 0x0004 flag is set. Depending on 0x0001 flag can be in Unix time_t or Windows FILETIME format.
atime	uint32 or uint64	Last access time. Present if 0x0008 flag is set. Depending on 0x0001 flag can be in Unix time_t or Windows FILETIME format.
mtime nanoseconds	uint32	Nanoseconds added to mtime value. Present if 0x0001, 0x0002 and 0x0010 flags are all set.
ctime nanoseconds	uint32	Nanoseconds added to ctime value. Present if 0x0001, 0x0004 and 0x0010 flags are all set.
atime nanoseconds	uint32	Nanoseconds added to atime value. Present if 0x0001, 0x0008 and 0x0010 flags are all set.

File version record

This record is used in archives created with -ver switch.

Size	vint	
Type	vint	0x04
Flags	vint	No file version flags are defined yet, so it is set to 0.
Version number	vint	File version number.

File system redirection record

Size	vint	
Type	vint	0x05
Redirection type	vint	0x0001 Unix symlink 0x0002 Windows symlink

		0x0003 Windows junction
		0x0004 Hard link
		0x0005 File copy
Flags	vint	0x0001 Link target is directory
Name length	vint	Length of link target name
Name	vint	Link target name in UTF-8 format without trailing zero

Unix owner record

Size	vint	
Type	vint	0x06
Flags	vint	0x0001 User name string is present 0x0002 Group name string is present 0x0004 Numeric user ID is present 0x0008 Numeric group ID is present
User name length	vint	Length of owner user name. Present if 0x0001 flag is set.
User name	? bytes	Owner user name in native encoding. Not zero terminated. Present if 0x0001 flag is set.
Group name length	vint	Length of owner group name. Present if 0x0002 flag is set.
Group name	? bytes	Owner group name in native encoding. Not zero terminated. Present if 0x0002 flag is set.
User ID	vint	Numeric owner user ID. Present if 0x0004 flag is set.
Group ID	vint	Numeric owner group ID. Present if 0x0008 flag is set.

Service data record

This record is used only by service headers to store additional parameters.

Size	vint	
Type	vint	0x07
Data	? bytes	Concrete contents of service data depends on service header type.

End of archive header

End of archive marker. RAR does not read anything after this header letting to use third party tools to add extra information such as a digital signature to archive.

Header CRC32	uint32	
Header size	vint	
Header type	vint	5
Header flags	vint	Flags common for all headers
End of archive flags	vint	0x0001 Archive is volume and it is not last volume in the set

Service headers

RAR uses service headers based on the [file header](#) data structure to store different supplementary information.

Archive comment header

Optional header storing the main archive comment. Contains CMT identifier in file name field. Placed before any file headers and after the main archive header. Comment data is stored in UTF-8 immediately after the archive comment header. Now RAR does not use compression for archive comments, so packed and unpacked data sizes in header are equal and they both define the comment data size. Compression method in header is set to 0.

Quick open header

Optional header storing the quick open record. Contains QO identifier in file name field. Placed after all file headers, but before the recovery record and end of archive header. It is possible to locate the quick open header with [locator record](#) in main archive header.

Quick open record data is stored immediately after the quick open header. RAR does not use compression for quick open data, so packed and unpacked data sizes in header are equal and they both define the quick open data size. Compression method in header is set to 0.

Quick open data is the array consisting of data cache structures. Every data cache structure stores a portion of archived data and has the following format:

Field	Size	Description
Structure CRC32	uint32	CRC32 of structure data starting from <i>Structure size</i> field.
Structure size	vint	Size of structure data starting from <i>Flags</i> field. This field must not be longer than 3 bytes in current implementation, resulting in 2 MB maximum size.
Flags	vint	Currently set to 0.
Offset	vint	Offset from beginning of quick open header to beginning of archive data cached in current structure. We can use this value to calculate the absolute position of archived data stored the current structure. It is guaranteed that absolute archive positions referred by data cache structures are always growing when going from beginning of structure array to end.
Data size	vint	Size of archive data stored in the current structure.
Data	? bytes	Archive data stored in the current structure.

Normally RAR uses the quick open data to store copies of file and service headers. It can store either all headers or only a part of them. If required header is missing in quick open data or if structure CRC32 is invalid, data are read from its original archive position.

Using the quick open data is optional. You can skip it completely and read only standard archive headers. But it is important to use the same access pattern when reading file names to display them to user and to extract files. Otherwise it would be possible to see one file name and extract another in case the quick open data and real archive data are intentionally created different. It could introduce a security threat. So if you use the quick open data when displaying the archive contents, use it when extracting. If you do not use it when displaying the archive contents, do not use it when extracting.